OS unit 1

Q1. Define an Operating System. Explain its Objectives and Functions.

Definition

An **Operating System (OS)** is system software that acts as an interface between the **user and the computer hardware**. It manages all hardware and software resources, controls the execution of programs, and provides services to application software.

Example: Windows, Linux, macOS, Android, iOS.

Objectives of an Operating System

1. Convenience

The operating system makes the computer easy to use by providing a user-friendly interface. Users can run applications without understanding the internal hardware.

2. Efficiency

It manages system resources such as CPU, memory, storage, and input/output devices efficiently to achieve better system performance.

3. Resource Management

The OS allocates and controls resources among different users and processes to avoid conflicts and ensure smooth operation.

4. Reliability

It detects and handles errors, protects data, and ensures the system continues to operate correctly.

5. Security

The operating system protects user data and programs from unauthorized access through authentication, passwords, and access permissions.

6. Ability to Evolve

Modern operating systems are designed to support new hardware and software without requiring major changes.

Functions of an Operating System

1. Process Management

The OS creates, schedules, executes, and terminates processes. It ensures efficient CPU utilization by managing multiple processes.

2. Memory Management

It allocates and deallocates memory to programs and keeps track of memory usage. It also prevents one program from interfering with another.

3. File Management

The operating system organizes files and directories, controls file access, and performs operations such as creating, deleting, renaming, and copying files.

4. Device Management

It manages input and output devices such as keyboards, printers, scanners, and disks using device drivers.

5. Security and Protection

The OS protects system resources through user authentication, access control, and data encryption.

6. User Interface

It provides a **Command Line Interface (CLI)** or **Graphical User Interface (GUI)** to allow users to interact with the computer.

7. Error Detection and Handling

The OS detects hardware and software errors, reports them, and takes corrective actions to maintain system stability.

8. Resource Allocation

It allocates CPU time, memory, storage, and I/O devices among multiple users and applications efficiently.

Q2. Explain the Evolution of Operating Systems.

Introduction

The **evolution of Operating Systems** refers to the development of operating systems over time to improve computer performance, resource utilization, and user convenience. As computer technology advanced, operating systems evolved from simple manual systems to modern, intelligent, and distributed systems.

---

1. Serial Processing Systems (1940s–1950s)

- Early computers had **no operating system**.
- Programs were loaded manually using switches or punched cards.
- Only one user could use the computer at a time.
- Execution was slow, and CPU remained idle while programs were loaded.

Advantages

- Simple architecture.
- Suitable for small programs.

Disadvantages

- Time-consuming.
- Poor CPU utilization.
- No automation.

---

2. Batch Operating Systems (1950s–1960s)

- Similar jobs were grouped into **batches** and executed automatically.
- No interaction between the user and the computer during execution.
- Jobs were processed one after another.

Advantages

- Reduced setup time.

- Better CPU utilization than serial processing.

Disadvantages

- No user interaction.
- Long waiting time.
- Difficult debugging.

---

3. Multiprogramming Operating Systems (1960s)

- Multiple programs were kept in memory simultaneously.
- When one program waited for I/O, the CPU executed another program.
- Increased CPU utilization and system efficiency.

Advantages

- Higher CPU utilization.
- Better throughput.
- Reduced idle time.

Disadvantages

- Complex memory management.
- Scheduling becomes difficult.

---

4. Time-Sharing Operating Systems (1970s)

- CPU time was divided into small **time slices**.
- Multiple users could interact with the computer simultaneously.
- Each user experienced quick response times.

Advantages

- Interactive computing.
- Supports multiple users.
- Faster response time.

Disadvantages

- Security concerns.
- Requires powerful hardware.

---

5. Multiprocessing Operating Systems

- Uses **two or more CPUs/processors** working together.
- Tasks are distributed among processors to improve performance.

Advantages

- High processing speed.
- Increased reliability.
- Better resource utilization.

Disadvantages

- Expensive hardware.
- Complex synchronization.

---

6. Distributed Operating Systems

- Multiple computers are connected through a network and work as a single system.
- Resources such as files, printers, and processors are shared.

Advantages

- Resource sharing.
- High reliability.
- Easy communication.

Disadvantages

- Complex design.
- Network dependency.

---

7. Real-Time Operating Systems (RTOS)

- Designed for applications where responses must be provided within a fixed time.
- Used in embedded systems, robotics, medical devices, and industrial control.

Advantages

- Fast response.
- High reliability.
- Predictable performance.

Disadvantages

- Expensive.
- Limited flexibility.

Q3. Explain the Basic Concepts of an Operating System.

The **basic concepts of an Operating System** are the fundamental components that help the operating system manage computer resources and execute programs. According to the syllabus, these concepts include **Processes, Files, and System Calls.**

---

1. Process

A **process** is a program that is currently being executed. It is an active entity that contains the program code, current activity, memory, and other resources required for execution.

Characteristics

- Has a unique Process ID (PID).
- Requires CPU time and memory.
- Can exist in different states such as **New, Ready, Running, Waiting,** and **Terminated**.

Example: When you open Google Chrome, each running tab may execute as a separate process.

---

2. File

A **file** is a collection of related information stored on a storage device. Files are used to permanently store data and programs.

Characteristics

- Each file has a unique name.
- Files are stored in directories (folders).
- The operating system manages file creation, deletion, reading, writing, copying, and renaming.

Example: report.docx, photo.jpg, music.mp3.

---

3. System Calls

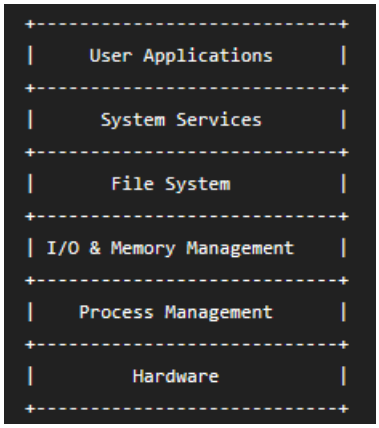
A **system call** is the interface through which a user program requests services from the operating system kernel. Since applications cannot directly access hardware, they use system calls to perform privileged operations.

Common Types of System Calls

- **Process Control:** Create process, terminate process, execute program.
- **File Management:** Create, open, read, write, close, and delete files.
- **Device Management:** Request or release I/O devices.
- **Information Maintenance:** Get system date, time, and process information.
- **Communication:** Exchange data between processes.

Example: Opening a file in a text editor causes the program to use a system call to access the file stored on disk.

Q4. Explain the Layered Structure of an Operating System with a neat diagram.



The **Layered Structure** is an operating system design in which the OS is divided into several layers. Each layer performs a specific function and provides services to the layer above it while using services from the layer below. This modular approach makes the operating system easier to design, maintain, and debug.

Working of Layered Structure

- **Hardware Layer:** The lowest layer consisting of CPU, memory, disks, and I/O devices.
- **Process Management:** Manages process creation, scheduling, synchronization, and termination.
- **Memory and I/O Management:** Allocates memory and manages input/output devices.
- **File System:** Organizes, stores, retrieves, and manages files and directories.
- **System Services:** Provides services such as system calls and utilities to application programs.
- **User Applications:** The topmost layer where users interact with software like browsers, editors, and media players.

Each layer communicates **only with the adjacent layer**, ensuring better modularity and reducing system complexity.

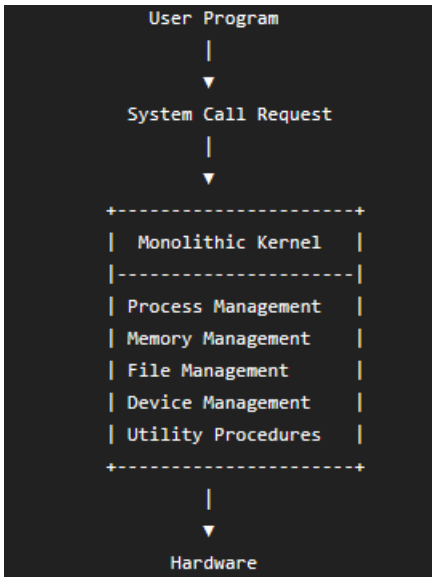
Advantages

1. **Modularity:** Each layer performs a specific function.
2. **Easy Maintenance:** Changes in one layer have minimal impact on other layers.
3. **Easy Debugging:** Errors can be identified and corrected layer by layer.
4. **Better Security:** Lower layers are protected from direct access by users.
5. **Easy Testing:** Each layer can be tested independently.

Disadvantages

1. **Reduced Performance:** Requests must pass through multiple layers, increasing execution time.
2. **Complex Design:** Proper division of functions into layers is difficult.
3. **Communication Overhead:** Every layer adds extra processing.
4. **Not Suitable for All Operating Systems:** Some OS functions require direct hardware access, making strict layering difficult.

Q5. Explain the Monolithic Structure of an Operating System with a neat diagram.



A **Monolithic Operating System** is one of the earliest operating system structures in which the **entire operating system works as a single large program (kernel)**. All operating system services such as process management, memory management, file management, and device management are combined into one kernel. There is **no clear separation between different services**, and all components share the same memory space.

Characteristics

- It is a **collection of procedures**.
- It has **no predefined structure**.
- There is **no information hiding**.
- All operating system services execute within the **same kernel space**.
- All modules can directly communicate with each other.

Working of Monolithic Operating System

1. A **user mode program** requests an operating system service through a **system call**.
2. The processor **traps the system call** and switches from **user mode to kernel mode**.
3. The required **service procedure** inside the kernel is executed.
4. Utility procedures are used whenever required by multiple service procedures.
5. After the service is completed, control returns to the user program by switching back to **user mode**.

Advantages

1. High execution speed due to direct communication between kernel components.
2. Fast execution of system calls.

3. Efficient resource management since all services are inside one kernel.
4. Simple communication between different operating system services.

Disadvantages

1. Large kernel size makes the system difficult to understand.
2. Difficult to debug and maintain.
3. A bug in one module may crash the entire operating system.
4. Poor modularity because there is no information hiding.

Q6. Compare Layered Structure and Monolithic Structure of Operating Systems.

Layered Structure	**Monolithic Structure**
OS is organized into **multiple layers**.	OS is a **single large kernel** containing all services.
Each layer performs a **specific function**.	All services are combined in one kernel.
Each layer communicates only with the **adjacent layer**.	All kernel modules communicate directly with each other.
Has **well-defined structure**.	Has **no predefined structure (Big Mess)**.
Provides **information hiding** between layers.	**No information hiding**.
Easier to **develop, test, and maintain**.	Difficult to **develop, debug, and maintain**.
More secure because layers are isolated.	Less secure because all modules share the same memory space.

Layered Structure	Monolithic Structure
Slower due to communication through multiple layers.	Faster because services communicate directly.
Errors are easier to locate and fix.	A bug in one module may crash the entire system.
Example: THE Operating System.	**Examples:** UNIX, Linux (Monolithic Kernel), MS-DOS.

Source: The comparison is based on the Layered Systems and Monolithic Systems sections in your PPT.

Q7. Differentiate between Process and Program.

Process	Program
A **process** is a program that is currently **executing**.	A **program** is a set of instructions stored on a storage device.
It is an **active entity**.	It is a **passive entity**.
Stored in **main memory (RAM)** during execution.	Stored on **secondary storage (disk)**.
Has a **Process ID (PID)**.	Does not have a PID.
Requires CPU time, memory, and other resources.	Does not require resources until executed.
Has different states such as **New, Ready, Running, Waiting, and Terminated**.	Has no execution states.
One program can create **multiple processes**.	A single program may have one or more processes during execution.
Example: Running Google Chrome.	**Example:** chrome.exe stored on disk.

8. Explain the Four Components of a Computer System.

A computer system consists of **four main components** that work together to perform tasks efficiently.

1. Hardware

Hardware provides the basic computing resources of the computer system. It includes:

- CPU (Central Processing Unit)
- Main Memory (RAM)
- Input/Output Devices
- Storage Devices

Hardware performs all physical operations in the computer.

---

2. Operating System (OS)

The Operating System controls and coordinates the use of hardware among various application programs and users. It acts as an interface between the user and the hardware.

Examples: Windows, Linux, macOS.

---

3. Application Programs

Application programs define how system resources are used to solve user problems.

Examples include:

- Word Processors
- Compilers
- Web Browsers
- Database Systems
- Video Games

---

4. Users

Users are the entities that interact with the computer system.

Users may include:

- People
- Machines
- Other Computers

9. Explain Operating System Services.

An Operating System provides various services to users and application programs for efficient execution.

Operating System Services

1. User Interface (UI)

Provides an interface through which users interact with the computer.

Types:

- Command Line Interface (CLI)
- Batch Interface
- Graphical User Interface (GUI)

2. Program Execution

Loads programs into memory, executes them, and terminates them after execution.

3. I/O Operations

Controls communication between the computer and input/output devices.

4. File System Manipulation

Creates, deletes, reads, writes, copies, and manages files and directories.

5. Communication

Allows data exchange between processes or between computers connected through a network.

6. Error Detection

Detects hardware and software errors and takes appropriate action.

7. Resource Allocation

Allocates CPU, memory, storage, and I/O devices among multiple users and processes.

8. Accounting

Keeps track of resource usage by users and processes.

9. Protection and Security

Protects system resources from unauthorized access and ensures data security.

---

10. Explain User View and System View of an Operating System.

An Operating System can be viewed from two different perspectives:

1. User View (Convenience)

- The operating system makes the computer easy to use.
- It provides a user-friendly interface.
- Users can run applications without knowing the hardware details.
- The main objective is **Convenience**.

---

2. System View (Efficiency)

- The operating system acts as a **resource manager**.
- It manages CPU, memory, storage, and I/O devices.
- It allocates resources efficiently among users and applications.
- The main objective is **Efficiency**.

User View	System View
Focuses on user convenience	Focuses on efficient resource management
Provides an easy-to-use interface	Manages hardware resources
Hides hardware complexity	Controls CPU, memory, files, and I/O

11. Explain Kernel, Microkernel, Monolithic Kernel, and Hybrid Kernel.

1. Kernel

The **Kernel** is the core component of an operating system. It manages hardware resources, memory, processes, and communication between hardware and software.

---

2. Microkernel

A **Microkernel** contains only the essential services such as process management, memory management, and communication. Other services like file systems and device drivers run in user space.

Advantages

- Better security
- Easy maintenance
- Improved reliability

---

3. Monolithic Kernel

A **Monolithic Kernel** contains all operating system services within a single kernel space. All components communicate directly with each other, resulting in high performance.

Examples: Linux, UNIX.

---

4. Hybrid Kernel

A **Hybrid Kernel** combines the features of both Monolithic and Microkernel architectures. It provides high performance while maintaining modularity.

Examples: Windows NT, macOS.

12. Differentiate between User Space and Kernel Space.

User Space	Kernel Space
User applications execute in this space.	The operating system kernel executes in this space.
It is the memory area reserved for user processes.	It is the memory area reserved for the operating system.
Cannot directly access hardware resources.	Has direct access to all hardware resources.
Cannot directly access kernel memory.	Can access both kernel memory and user memory.
Uses **system calls** to request services from the kernel.	Provides services requested through system calls.
Executes with limited privileges.	Executes with full (privileged) access.
A crash in user space usually affects only that application.	A crash in kernel space may crash the entire operating system.
More secure because applications are isolated from the kernel.	Less isolated, since it controls the entire system.
Examples: Chrome, MS Word, VLC Media Player.	Examples: Process Scheduler, Memory Manager, File System, Device Drivers.
Main purpose is to run user applications safely.	Main purpose is to manage hardware and system resources efficiently.

Note: RAM is divided into **User Space** and **Kernel Space**. User processes run in User Space, while the operating system kernel runs in Kernel Space. User applications access kernel services through **system calls**.

13. Differentiate between User Mode and Kernel Mode.

User Mode	Kernel Mode
Applications execute in User Mode.	The operating system executes in Kernel Mode.
Has limited privileges.	Has full privileges.
Cannot directly access hardware.	Can directly access hardware.
Cannot execute privileged CPU instructions.	Can execute all CPU instructions.
Cannot directly access kernel memory.	Can access all memory locations.
Uses **system calls** to request operating system services.	Executes system calls and provides OS services.
Failure usually affects only the application.	Failure may crash the entire operating system.
Provides security by restricting application access.	Responsible for resource management and system protection.
Examples: Web browsers, text editors, games.	Examples: Process scheduling, memory management, device drivers.
Processor switches to Kernel Mode during interrupts or system calls and returns to User Mode after completing the service.	Processor starts in Kernel Mode during boot and handles all privileged operations.

14. Explain Bootstrap Program.

Introduction

A **Bootstrap Program** is a small program that is executed whenever a computer is **powered on or restarted**. It is responsible for initializing the computer system and loading the operating system into memory.

Features of Bootstrap Program

1. It is **loaded at power-up or reboot** of the computer.
2. It is **typically stored in ROM (Read Only Memory)**.
3. It **initializes all aspects of the computer system**, such as CPU, memory, and input/output devices.
4. It **locates the Operating System Kernel** stored on the storage device.
5. It **loads the kernel into the main memory (RAM)**.
6. After loading the kernel, it **starts the execution of the Operating System**.

Working of Bootstrap Program

1. The computer is switched **ON**.
2. The Bootstrap Program stored in **ROM** starts executing.
3. It checks and initializes the hardware components.
4. It searches for the Operating System Kernel.
5. The kernel is loaded into RAM.
6. Control is transferred to the Operating System, and the system becomes ready for use.

15. Explain Storage Device Hierarchy.

Introduction

The **Storage Device Hierarchy** is the arrangement of different storage devices based on their **speed, capacity, cost, and access time**. Storage devices at the top are faster but smaller and more expensive, while those at the bottom are slower but provide larger storage at a lower cost.

---

1. Main Memory (RAM)

- Programs must be loaded into **Main Memory (RAM)** before they can be executed.
- It provides **Random Access** to data.
- It is **volatile**, meaning data is lost when power is switched off.
- The CPU interacts with RAM through **load** and **store** instructions.
- It has **limited storage capacity**.

---

2. Secondary Storage

- Secondary storage acts as an **extension of main memory**.
- It provides **large non-volatile storage capacity**.
- Data remains stored even when power is turned off.
- **Magnetic disks** are commonly used secondary storage devices.
- The disk surface is divided into **tracks** and **sectors**.
- The **disk controller** manages communication between the disk and the computer.

Characteristics of Storage Hierarchy

- Small storage devices are **faster** in operation.
- Large storage devices provide **higher storage capacity**.
- Faster storage devices are **more expensive**.

- Slower storage devices are **cheaper**.
- As **storage size increases, access time also increases**.
- **Speed and cost per bit increase** towards the top of the hierarchy.

16. Explain Computer System Operation.

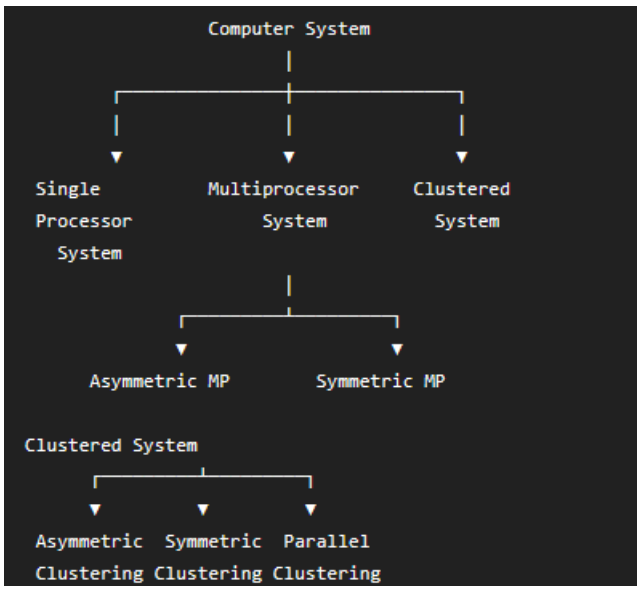
Introduction

Computer System Operation describes how the CPU, memory, and I/O devices work together to execute programs efficiently. The operating system coordinates these components and manages the transfer of data between them.

Computer System Operation

1. **I/O devices and the CPU can execute concurrently**, allowing the processor and input/output devices to work simultaneously.
2. **Each device controller** is responsible for controlling a particular type of device (such as a keyboard, printer, or disk).
3. Every **device controller has a local buffer** to temporarily store data during data transfer.
4. The **CPU transfers data** between the main memory and the local buffer of the device controller.
5. During an **input operation**, data is transferred from the input device to the local buffer of the device controller.
6. When the I/O operation is completed, the **device controller generates an interrupt** to inform the CPU that the operation has finished

17. Explain Computer System Architecture.



Introduction

Computer System Architecture refers to the organization of processors in a computer system. Based on the **number of general-purpose processors**, computer systems are classified into three types:

1. Single Processor Systems
2. Multiprocessor Systems
3. Clustered Systems

---

1. Single Processor System

- Uses **only one general-purpose processor**.
- May also contain special-purpose processors.
- The operating system communicates only with the general-purpose processor.

---

2. Multiprocessor System

A multiprocessor system contains **two or more general-purpose processors** working together.

It is also known as:

- Parallel System
- Tightly Coupled System

Advantages

- Increased Throughput
- Economy of Scale
- Increased Reliability (Graceful Degradation / Fault Tolerance)

Types

- **Asymmetric Multiprocessing (AMP):** Master-Slave architecture.
- **Symmetric Multiprocessing (SMP):** All processors are equal (Peers).

---

3. Clustered System

A **Clustered System** consists of **two or more independent computer systems** connected through a LAN and sharing storage.

Advantages

- High Availability
- Better Reliability

Types

- Asymmetric Clustering
- Symmetric Clustering
- Parallel Clustering

18. Explain Shell.

Introduction

A **Shell** is a **command interpreter** that acts as the primary interface between the **user and the operating system**. It accepts commands from the user and passes them to the operating system for execution.

Features of Shell

1. It is the **Unix command interpreter**.
2. It acts as the **primary interface** between the user and the operating system.
3. It accepts commands from the user through the **terminal**.
4. It displays the output of executed commands.
5. It begins with the **$ prompt** to accept user input.
6. Different types of shells are available:
    - **sh**
    - **csh**
    - **ksh**
    - **bash**